

Table of Contents

Abstract.....	i
1. Introduction.....	1
2. Description and Explanation	2
3. Source Code	3
4. Screenshots	4
.4.1 The Diagram (Heading 2).....	4
.4.2 Simulation Trace (Waveform) (Heading 2).....	4
5. Tasks Distributions	9
6. References	10

1. Introduction

The Arithmetic Logic Unit (ALU) is a core component of the Central Processing Unit (CPU) responsible for performing arithmetic and logical operations on binary data. The accuracy and efficiency of the ALU have a direct impact on the overall system performance.

The problem addressed by this project is the need to design a flexible and efficient ALU capable of executing a set of essential operations, including:

- Arithmetic operations (addition, subtraction, multiplication, division)
- Logical operations (AND, OR, XOR, NOT)
- Bit-level data manipulation (shifting, masking)
- Comparison operations for binary values

A well-structured ALU that supports these functions enhances the processor's ability to handle various computational tasks effectively, which is the main goal of this project.

2. Description and Explanation

The **Arithmetic Logic Unit (ALU)** is designed to perform arithmetic and logical operations as part of the processor's execution cycle. The ALU's design involves key components like multiplexers, full adders, shifters, and Boolean gates.

1. Basic Operations:

- **Addition:** Implemented using a ripple-carry adder or a carry-lookahead adder.
- **Subtraction:** Implemented using two's complement.
- **Multiplication:** Implemented using shift-and-add algorithms or Booth's algorithm.
- **Division:** Implemented using restoring or non-restoring division algorithms.

2. Logical Operations:

Logic gates like **AND**, **OR**, **NOT**, **XOR** are used to perform bitwise comparisons of the input operands.

3. Flags:

Status flags are generated after each operation:

- **Zero Flag (Z):** Set if the result is zero.
- **Carry Flag (C):** Set to detect overflows in unsigned arithmetic.

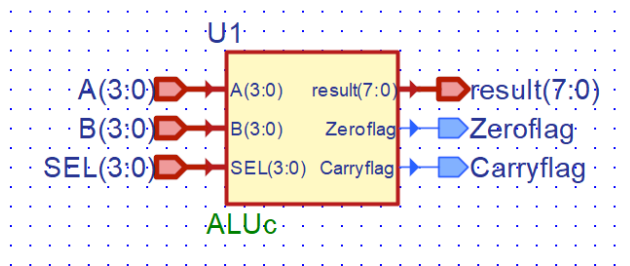
These flags are stored in a status register and are used to control decision-making in program flow.

3. Source Code

```
27 module ALUc (A, B, SEL, result, Zeroflag, Carryflag);
28
29 input [3:0] A, B;
30 input [3:0] SEL;
31 output reg [7:0] result;
32 output reg Zeroflag;
33 output reg Carryflag;
34
35 reg [4:0] temp;
36
37 always @ (A or B or SEL) begin
38     Carryflag = 0;
39     case (SEL)
40         4'b0000: begin // Addition
41             temp = A + B;
42             result = {3'b000, temp[3:0]};
43             Carryflag = temp[4];
44         end
45         4'b0001: begin // Subtraction
46             temp = A - B;
47             result = {3'b000, temp[3:0]};
48             Carryflag = temp[4];
49         end
50         4'b0010: result = A * B; //Multiplication
51         4'b0011: begin
52             if (B != 0)
53                 result = A / B; //Division
54             else
55                 result = 8'b00000000;
56         end
57         4'b0100: result = {4'b0000, A & B}; //AND
58         4'b0101: result = {4'b0000, A | B}; //OR
59         4'b0110: result = {4'b0000, ~A}; //NOT A
60         4'b0111: result = {4'b0000, A ^ B}; //XOR
61         4'b1000: result = {4'b0000, ~B}; //NOT B
62         default: result = 8'b00000000;
63     endcase
64
65     if (result == 8'b00000000)
66         Zeroflag = 1'b1;
67     else
68         Zeroflag = 1'b0;
69 end
70
71 endmodule
72
```

4. Screenshots

4.1. The Diagram



4.2. Simulation Trace

1. Basic Operations: Addition:

Signal name	Value	8	16	24	32	40	48	56
JL temp	0C						0C	
A	8						8	
A[3]	1							
A[2]	0							
A[1]	0							
A[0]	0							
B	4						4	
B[3]	0							
B[2]	1							
B[1]	0							
B[0]	0							
SEL	0						0	
SEL[3]	0							
SEL[2]	0							
SEL[1]	0							
SEL[0]	0							
result	0C						0C	
result[7]	0							
result[6]	0							
result[5]	0							
result[4]	0							
result[3]	1							
result[2]	1							
result[1]	0							
result[0]	0							
Zeroflag	0							
Carryflag	0							

nameValue

A

8

B

4

SEL

0

result

0C

Zeroflag

0

Carryflag

0

temp

0C

Subtraction:

Hierarchy		Signal name	Value	112	120	128	136	144	152
ALUc		temp	04	04					
A		A	8	8					
		A[3]	1						
		A[2]	0						
		A[1]	0						
		A[0]	0						
B		B	4	4					
		B[3]	0						
		B[2]	1						
		B[1]	0						
		B[0]	0						
SEL		SEL	1	1					
		SEL[3]	0						
		SEL[2]	0						
		SEL[1]	0						
		SEL[0]	1						
result		result	04	04					
		result[7]	0						
		result[6]	0						
		result[5]	0						
		result[4]	0						
		result[3]	0						
		result[2]	1						
		result[1]	0						
		result[0]	0						
		Zeroflag	0						
		Carryflag	0						

Name	Value
A	8
B	4
SEL	1
result	04
Zeroflag	0
Carryflag	0
(x) temp	04

Multiplication:

ALUC

hierarchy

ALUC

Signal name	Value	608	616	624	632	640	648	656
temp	0C	0C						
A	2	2						
A[3]	0							
A[2]	0							
A[1]	1							
A[0]	0							
B	4	4						
B[3]	0							
B[2]	1							
B[1]	0							
B[0]	0							
SEL	2	2						
SEL[3]	0							
SEL[2]	0							
SEL[1]	1							
SEL[0]	0							
result	08	08						
result[7]	0							
result[6]	0							
result[5]	0							
result[4]	0							
result[3]	1							
result[2]	0							
result[1]	0							
result[0]	0							
Zeroflag	0							
Carryflag	0							

Name	Value
A	2
B	4
SEL	2
result	08
Zeroflag	0
Carryflag	0
(x) temp	0C

Division:

Hierarchy		Signal name	Value	808	816	824	832	840	848	856
ALUc		temp	0C						0C	
A		A	8						8	
A[3]		A[3]	1							
A[2]		A[2]	0							
A[1]		A[1]	0							
A[0]		A[0]	0							
B		B	4						4	
B[3]		B[3]	0							
B[2]		B[2]	1							
B[1]		B[1]	0							
B[0]		B[0]	0							
SEL		SEL	3						3	
SEL[3]		SEL[3]	0							
SEL[2]		SEL[2]	0							
SEL[1]		SEL[1]	1							
SEL[0]		SEL[0]	1							
result		result	02						02	
result[7]		result[7]	0							
result[6]		result[6]	0							
result[5]		result[5]	0							
result[4]		result[4]	0							
result[3]		result[3]	0							
result[2]		result[2]	0							
result[1]		result[1]	1							
result[0]		result[0]	0							
Zeroflag		Zeroflag	0							
Carryflag		Carryflag	0							

Name	Value
A	8
B	4
SEL	3
result	02
Zeroflag	0
Carryflag	0
temp	0C

2. Logical Operations: AND

Hierarchy		Signal name	Value	912	920	928	936	944	952
ALUc		temp	0C						0C
A		A	8						8
A[3]		A[3]	1						
A[2]		A[2]	0						
A[1]		A[1]	0						
A[0]		A[0]	0						
B		B	4						4
B[3]		B[3]	0						
B[2]		B[2]	1						
B[1]		B[1]	0						
B[0]		B[0]	0						
SEL		SEL	4						4
SEL[3]		SEL[3]	0						
SEL[2]		SEL[2]	1						
SEL[1]		SEL[1]	0						
SEL[0]		SEL[0]	0						
result		result	00						00
result[7]		result[7]	0						
result[6]		result[6]	0						
result[5]		result[5]	0						
result[4]		result[4]	0						
result[3]		result[3]	0						
result[2]		result[2]	0						
result[1]		result[1]	0						
result[0]		result[0]	0						
Zeroflag		Zeroflag	1						
Carryflag		Carryflag	0						

Name	Value
A	8
B	4
SEL	4
result	00
Zeroflag	1
Carryflag	0
temp	0C

OR:

Hierarchy	
ALUc	

Name	Value
A	8
B	4
SEL	5
result	0C
Zeroflag	0
Carryflag	0
(x) temp	0C

Signal name	Value	1408	1416	1424	1432	1440	1448	1456
temp	0C							0C
A	8							8
A[3]	1							
A[2]	0							
A[1]	0							
A[0]	0							
B	4							4
B[3]	0							
B[2]	1							
B[1]	0							
B[0]	0							
SEL	5							5
SEL[3]	0							
SEL[2]	1							
SEL[1]	0							
SEL[0]	1							
result	0C							0C
result[7]	0							
result[6]	0							
result[5]	0							
result[4]	0							
result[3]	1							
result[2]	1							
result[1]	0							
result[0]	0							
Zeroflag	0							
Carryflag	0							

XOR:

Hierarchy	
ALUc	

Name	Value
A	8
B	4
SEL	7
result	0C
Zeroflag	0
Carryflag	0
(x) temp	0C

Signal name	Value	1608	1616	1624	1632	1640	1648	1656
temp	0C							0C
A	8							8
A[3]	1							
A[2]	0							
A[1]	0							
A[0]	0							
B	4							4
B[3]	0							
B[2]	1							
B[1]	0							
B[0]	0							
SEL	7							7
SEL[3]	0							
SEL[2]	1							
SEL[1]	1							
SEL[0]	1							
result	0C							0C
result[7]	0							
result[6]	0							
result[5]	0							
result[4]	0							
result[3]	1							
result[2]	1							
result[1]	0							
result[0]	0							
Zeroflag	0							
Carryflag	0							

6. References

<https://github.com/SarwanShah/8-bit-ALU-Using-Logic-Gates-2017>

<https://github.com/Chandutv0808/-ARITHMETIC-LOGIC-UNIT-ALU->

<https://www.slideshare.net/slideshow/vlsi-design-final-project-32-bit-alu/64913691>

<https://engr210.github.io/projects/P6.html>